

MLE-STAR: feature/mcts-skrub

Understanding the MCTS + skrub integration

This document explains the new files introduced in the feature/mcts-skrub branch: a pure MCTS engine, a skrub bridge for action spaces and rollouts, staged pipeline construction, and the associated tests and documentation.

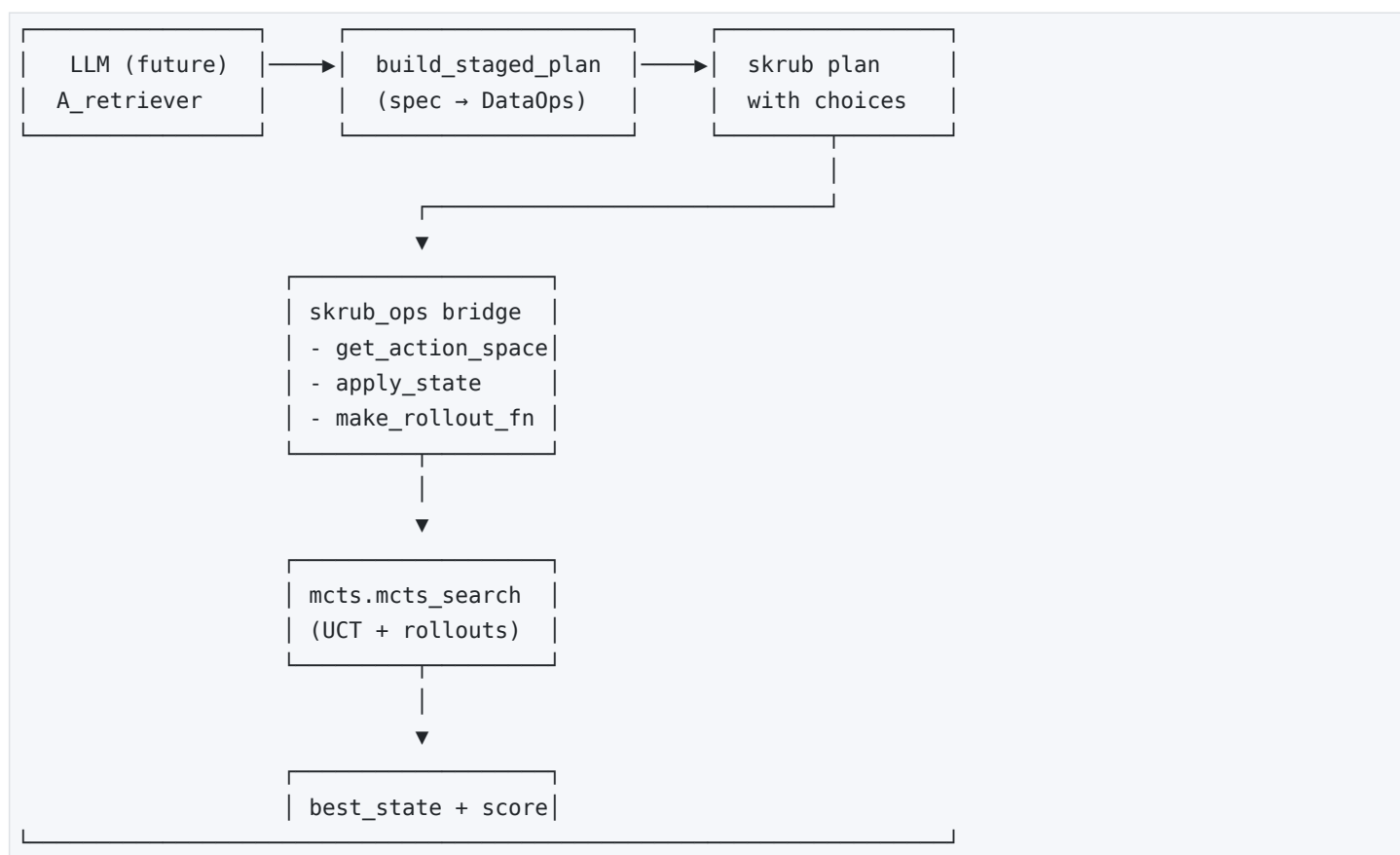
Generated: 2026-06-15

1. What is skrub?

skrub is a Python library for building machine-learning pipelines over messy, heterogeneous tabular data: high-cardinality categoricals, dates, free text, and related tables. Unlike plain scikit-learn, skrub lets you declare a pipeline as a DAG of DataOps nodes. Some of those nodes are choices: `choose_from`, `choose_int`, `choose_float`. A single pipeline object can therefore represent many concrete configurations.

The central idea of feature/mcts-skrub is to treat that choice graph as the search space for Monte Carlo Tree Search (MCTS). The LLM proposes the structure; MCTS navigates the discrete choices and numeric discretizations.

1.1 High-level architecture



2. machine_learning_engineering/mcts.py

mcts.py is a dependency-free implementation of Monte Carlo Tree Search with the UCT selection rule. It does not know anything about skrub, LLMs, or the network. Its only inputs are a root state (a dictionary), a finite action space (also a dictionary), and a rollout function that maps a state to a scalar reward in [0, 1].

2.1 Core classes and functions

MCTSNode

Each node stores a compact state dictionary, visit count N , cumulative reward Q , parent pointer, and a list of children. States are made hashable through `state_key()`, which sorts keys recursively so that equivalent states collide regardless of insertion order.

uct(c=0.5)

```
score = Q/N + c * sqrt(log(parent.N) / N)
```

The UCT formula balances exploitation (high average reward) with exploration (low visit count). Unvisited children are assigned $+\infty$ so the tree always expands an unseen node before re-visiting one.

select, expand, backpropagate

- **select**: descend from the root, always choosing the child with the highest UCT score, until a leaf is reached.
- **expand**: create child nodes by changing exactly one choice from the parent state. A global `tried_states` set prevents duplicates and cycles.
- **backpropagate**: walk up to the root and update N and Q with the reward obtained by the rollout.

mcts_search(...)

Main loop. It accepts an optional root and `tried_states` so the tree can be persisted across calls. It returns the best state found, its score, and the root node. Two utilities, `to_dot()` and `print_tree()`, help visualize the search tree.

Note: This is Optimization-MCTS, not Game-MCTS. There are no terminal states; every node is a complete configuration that can be evaluated directly.

3. machine_learning_engineering/skrub_ops.py

skrub_ops.py is the bridge between skrub pipelines and the MCTS engine. It is divided into introspection helpers, state application/evaluation helpers, staged plan builders, and ablation utilities.

3.1 Track A — Graph introspection

- `get_choices(plan)`: returns every `choose_*` node in the DAG.
- `get_action_space(plan, n_numeric_options=4)`: converts choices into a finite action space; numeric choices are discretized.
- `get_state(plan)`: returns the currently active choices as a compact dictionary.
- `get_default_state(plan)`: resets the plan to its default options and returns that state.
- `find_node(plan, name)`: locates a named node in the skrub DAG.

3.2 Track B — Applying states and evaluating

`apply_state(plan, state)`

Resets the plan to defaults and then sets each named choice to the value provided. This prevents leakage: values set by previous configurations cannot influence the new one. It also validates that every name and option exists.

`make_rollout_fn(plan, df, ...)`

Returns a function `rollout(state) -> float`. Internally it applies the state, samples a deterministic subset of the dataframe, runs `plan.skb.cross_validate(environment=...)`, and returns the mean test score. If anything fails it returns 0.0 so the search stays robust.

`evaluate_full(plan, state, df)`

Same as a rollout but uses the full dataset, does not swallow exceptions, and is meant for the final evaluation of the best configuration found by MCTS.

3.3 Staged plan construction

`build_staged_plan(spec, df, aux_tables=...)` lets an LLM or a human define a pipeline through a stage menu rather than writing skrub code by hand. Supported stages:

```
assemble → clean → encode → scale → feature_eng → model  
(optional) (optional) (optional) (optional) (optional) (required)
```

Each optional stage can be skipped (None). The model stage always has at least one default learner so that even a partially built plan is executable. This allows MCTS to search not only hyperparameters but also whether to include, for example, scaling or feature engineering.

3.4 Ablation helpers

- `run_ablation(plan, node_name, df, base_state)`: evaluates every option of a single choice while keeping all others fixed.
- `pick_target_node(ablation_results)`: selects the choice with the largest score variance, i.e., the most promising node to refine.

Note: Ablation is useful for diagnostics and for guiding the LLM: instead of searching the entire combinatorial space blindly, you can first identify which choices matter most.

4. Fixtures

4.1 tests/fixtures/golden_plan.py

A hand-written skrub plan for a synthetic binary classification dataset. It exposes choices for the encoder (GapEncoder vs MinHashEncoder), the model family (GBM vs RF), and conditional hyperparameters. It is used for unit tests and as a stable benchmark while developing the MCTS bridge.

4.2 tests/fixtures/staged_plan.py

Two stories for the staged builder:

- Flat/enrichment: the target depends on an interaction ($x_1 * x_2$). A linear model is weak until PolynomialFeatures is enabled.
- Relational: the target depends on an aggregate from an auxiliary table. The assemble stage with AggJoiner raises the score from near-chance to strong.

5. Test suite

5.1 test_mcts.py

Tests the MCTS engine in isolation using a fake reward that rewards matching a hidden target state. Covers UCT behavior, expansion without duplicates, backpropagation, search convergence, persistence across calls, failure tolerance, and DOT/ASCII output.

5.2 test_skrub_ops.py

Tests the skrub bridge: action-space coverage, state round-trips, validation of invalid states, deterministic rollouts, full evaluation, ablation, and a small end-to-end MCTS run over the golden plan.

5.3 test_staged.py

Tests build_staged_plan: menu-to-action-space translation, default baseline, leak-safe partial states, score improvement when enrichment stages are enabled, and the relational assemble story.

5.4 test_integration.py

The composite golden-path test. It wires mcts.py to skrub_ops.py through the golden plan and asserts structural properties of the resulting search tree:

- skrub state round-trips correctly to MCTS state and back.
- Every action-space option is applicable.
- expand produces single-edit neighbors.
- The tree is well-formed (unique nodes, consistent parent/child links, $\text{child.N} \leq \text{parent.N}$).
- Per-iteration statistics update by exactly one visit and one reward in $[0, 1]$.

6. Documentation

- docs/mcts-uct.md: team guide to MCTS/UCT. Explains $c=0.5$, Optimization-MCTS vs Game-MCTS, transpositions, cycles, and reading pointers.
- docs/pipeline-stages.md: taxonomy of tabular/relational pipeline stages, what build_staged_plan supports, and what is still pending.
- docs/demo-datasets.md: recommended skrub datasets for a public demo and the current implementation status.

7. What is still missing?

The branch provides the search and evaluation infrastructure, but the LLM connector is not yet implemented. The remaining pieces are:

- A_retriever: an LLM agent that turns a task description into the spec dictionary accepted by build_staged_plan.
- A_leakage: an LLM agent that reviews staged plans for data leakage, especially AggTarget aggregations.
- Outer orchestrator: a loop that asks the LLM for a plan, runs MCTS, evaluates the best state, and produces the final executable script.
- Integration with the existing OpenAI port: our MVP in feature/issue-1.5-openai-port currently generates raw Python code. This branch proposes an alternative path where the LLM emits a structured skrub plan and MCTS optimizes inside it.

Note: No decision is required now; this document only records what the branch already contains.